Experiment No. 1

Title: Design a Responsive Registration Form using Bootstrap and Perform Client-Side Validation using JavaScript

Aim:

To design a responsive reader registration form using the Bootstrap 5 framework and implement robust client-side validation using the HTML5 Constraint Validation API and vanilla JavaScript, ensuring that user input is verified prior to submission.

Tools Used:

- HTML5 semantic form controls (input, select, textarea, date, checkbox)
- Bootstrap 5.3 CSS framework (12-column responsive grid, cards, input groups, modal)
- Vanilla JavaScript (ES6+) for DOM manipulation and constraint validation
- Regular expressions for Indian mobile numbers and pincodes
- Google Chrome Headless automated test runner for screenshot evidence capture

Methodology:

Bootstrap 5 Responsive Grid → HTML5 Constraint Markup → Vanilla JS Event Listeners → Real-time DOM Feedback → Modal Summary → Multi-Device Verification

The form was developed using a mobile-first 12-column responsive grid. Each input control is mapped to specific validation rules: mandatory required checks, minimum lengths, regex pattern matching, and custom logic for birthdate age constraints (minimum 13 years) and password complexity. Client-side event listeners on input and blur provide instantaneous feedback through Bootstrap is-valid and is-invalid classes. On submission, event.preventDefault() intercepts page reload, focuses the first invalid element if any errors exist, or displays a modal summary with all entered details.

Theory:

Client-side validation is a critical element of modern web usability and security. It ensures that user inputs adhere to required formats, bounds, and business invariants before transmitting payloads to backend servers. HTML5 provides built-in constraint validation through attributes such as required, minlength, maxlength, pattern, and specialized input types like email, tel, and date.

The JavaScript Constraint Validation API augments standard HTML5 attributes through programmatic methods including element.checkValidity(), form.checkValidity(), and element.setCustomValidity(). Calling setCustomValidity() with a non-empty string flags the input as invalid and customizes the browser's error message, whereas passing an empty string resets the validation state.

Bootstrap 5 provides visual validation classes (.is-valid, .is-invalid, and .was-validated) that pair with .valid-feedback and .invalid-feedback elements. When combined with form.addEventListener('submit', (e) => e.preventDefault()), developers can build intuitive, responsive forms that enhance user experience while minimizing unnecessary network round-trips.

Case Study Context:

Kitab Ghar, Badlapur — a neighbourhood library and book-rental subscription platform. Readers sign up for library cards to rent fiction, classics, non-fiction, and children's literature with doorstep delivery

across Badlapur and nearby Thane district localities. The registration interface collects personal credentials, delivery addresses, pincodes, age verification, and membership plan preferences.

A dedicated standalone demonstration module was built at experiments/exp1/ to demonstrate the exact curriculum requirement (pure Bootstrap 5 grid + CSS components + vanilla JavaScript validation) independently from the React SPA application, while sharing the identical Kitab Ghar design identity.

Functions / Methods / Technologies Used:

- document.getElementById() and querySelectorAll() for DOM element access
- addEventListener('input'), addEventListener('blur'), and addEventListener('change') for live validation
- HTMLInputElement.checkValidity() and HTMLSelectElement.checkValidity()
- HTMLInputElement.setCustomValidity() for custom password mismatch and age rules
- Element.classList.add() and Element.classList.remove() for dynamic Bootstrap styling
- Event.preventDefault() and Event.stopPropagation() for form submission interception
- new window.bootstrap.Modal(modalEl).show() for registration confirmation

Procedure:

The form was checked first with blank fields and then with invalid email, mobile number, age, pincode, password, and confirmation values. A completed reader form was finally submitted at desktop and mobile widths to confirm both validation behaviour and responsive layout.

1. Navigate to experiments/exp1/ and inspect index.html, styles.css, and script.js.
2. Structure the HTML using Bootstrap 5 container, row, col-12, col-md-6, and card classes.
3. Configure HTML5 attributes: required, autocomplete, pattern='[6-9][0-9]{9}' for mobile, and pattern='[0-9]{6}' for pincode.
4. Calculate maximum allowable date of birth in script.js to enforce the 13-year age threshold (dob.max = maxDobDate).
5. Attach input and blur event listeners to all input elements to evaluate validateField() in real-time.
6. Implement updatePasswordMeter() to score password complexity across 4 criteria (length, uppercase, lowercase, digits).
7. Attach the submit event listener on registration-form, invoking event.preventDefault() to suppress page reloads.
8. Execute checkValidity() and setCustomValidity() across all fields, accumulating any failure states.
9. If errors exist, focus the first failing element and render the danger alert banner.
10. Upon successful validation, compile reader details into modal-summary and trigger registrationModal.show().
11. Verify responsive design at 1280px (desktop dual-column) and 375px (mobile single-column) viewports.

Code:

Code Block 1.1: Field Validation Logic with Constraint Validation API

File: `experiments/exp1/script.js` (Lines 24–56) — Evaluates field correctness and applies setCustomValidity() and Bootstrap CSS classes.

```
function validateField(field) {
  if (!field) return true;
  let isValid = true;
  const value = (field.value || "").trim();
  if (field.id === "dob") {
    if (!value) return false;
    const birthDate = new Date(value);
    let age = today.getFullYear() - birthDate.getFullYear();
    const monthDiff = today.getMonth() - birthDate.getMonth();
    if (monthDiff < 0 || (monthDiff === 0 && today.getDate() < birthDate.getDate())) age--;
    isValid = age >= 13;
  } else if (field.id === "password") {
    isValid = value.length >= 8 && /[a-z]/.test(value) && /[A-Z]/.test(value) && /\d/.test(value);
    field.setCustomValidity(isValid ? "" : "Password must be at least 8 characters with uppercase, lowercase
and a digit.");
  } else if (field.id === "confirmPassword") {
    isValid = value.length > 0 && value === passwordInput.value;
    field.setCustomValidity(isValid ? "" : "Passwords do not match.");
  } else if (field.id === "terms") {
    isValid = field.checked;
  } else {
    isValid = field.checkValidity();
  }
  if (isValid) {
    field.setCustomValidity("");
    field.classList.remove("is-invalid");
    field.classList.add("is-valid");
  } else {
    field.classList.remove("is-valid");
    field.classList.add("is-invalid");
  }
  return isValid;
}
```

Code Block 1.2: Form Submission Handler & Modal Integration

File: `experiments/exp1/script.js` (Lines 96–129) — Suppresses page reload, aggregates field validity, and opens confirmation modal on success.

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  event.stopPropagation();
  let allValid = true;
  let firstInvalidField = null;
  inputElements.forEach((input) => {
    const fieldValid = validateField(input);
    if (!fieldValid && !firstInvalidField) firstInvalidField = input;
    if (!fieldValid) allValid = false;
  });
  if (!allValid) {
    formAlert.className = "alert alert-danger mb-3";
    formAlert.textContent = "Please review and correct the highlighted fields before submitting.";
    if (firstInvalidField) firstInvalidField.focus();
    return;
  }
  formAlert.className = "alert alert-success mb-3";
  formAlert.textContent = "All reader details are valid! Preparing registration confirmation...";
  modalSummary.innerHTML = `<p class="mb-1"><strong>Reader:</strong> ${form.fullName.value} |
${form.email.value}</p><p class="mb-1"><strong>Contact:</strong> +91 ${form.mobile.value} |
${form.city.value} (${form.pincode.value})</p>`;
  if (registrationModal) registrationModal.show();
});
```

Output:

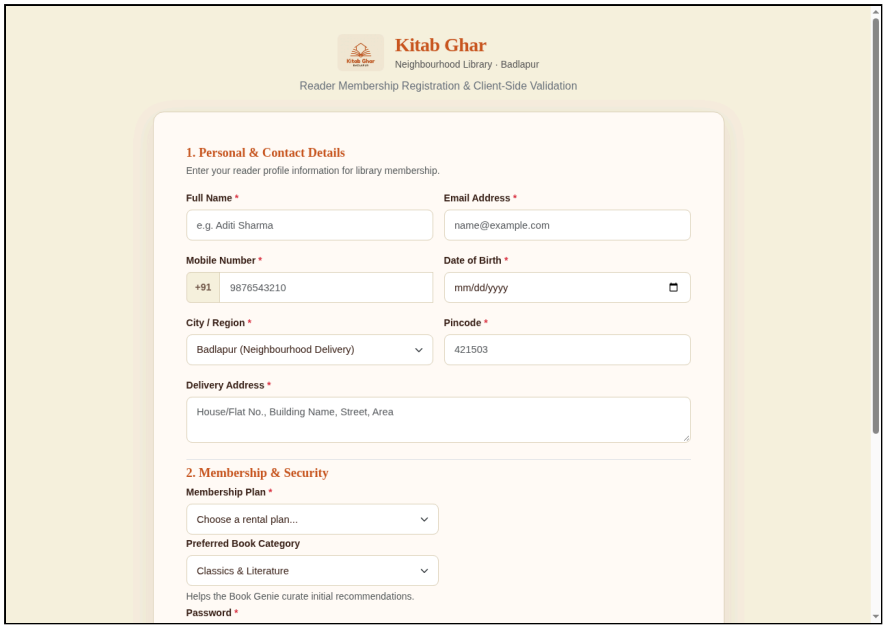


Figure 1.1: Kitab Ghar Reader Registration Form — Empty State

This screenshot shows the initial, unpopulated state of the Kitab Ghar reader registration form loaded at a desktop width of 1280px. The interface highlights the Bootstrap 5 two-column grid layout (col-md-6 responsive columns), clear sectional groupings for personal and library details, and prominent branding for the Badlapur neighbourhood library. The form is rendered inside a Bootstrap card component with appropriate spacing, and all input fields display placeholder text guiding the user on the expected format. No validation messages are visible at this stage, confirming the clean default state.

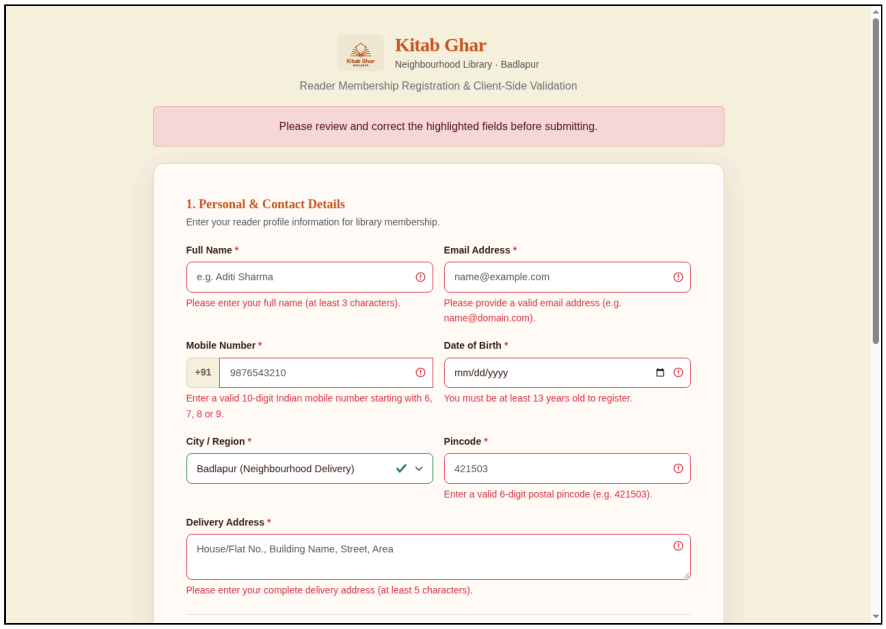


Figure 1.2: Empty Form Submission — Constraint Validation Feedback

This screenshot shows the result of submitting the empty registration form without populating mandatory fields. The JavaScript submit handler intercepts the event via preventDefault(), evaluates checkValidity() on all inputs, applies the Bootstrap is-invalid class, and displays red error messages beneath every required field while focusing the first invalid input. The form visually demonstrates how HTML5 constraint validation and Bootstrap feedback classes work together to provide immediate, accessible error guidance without any server round-trip.

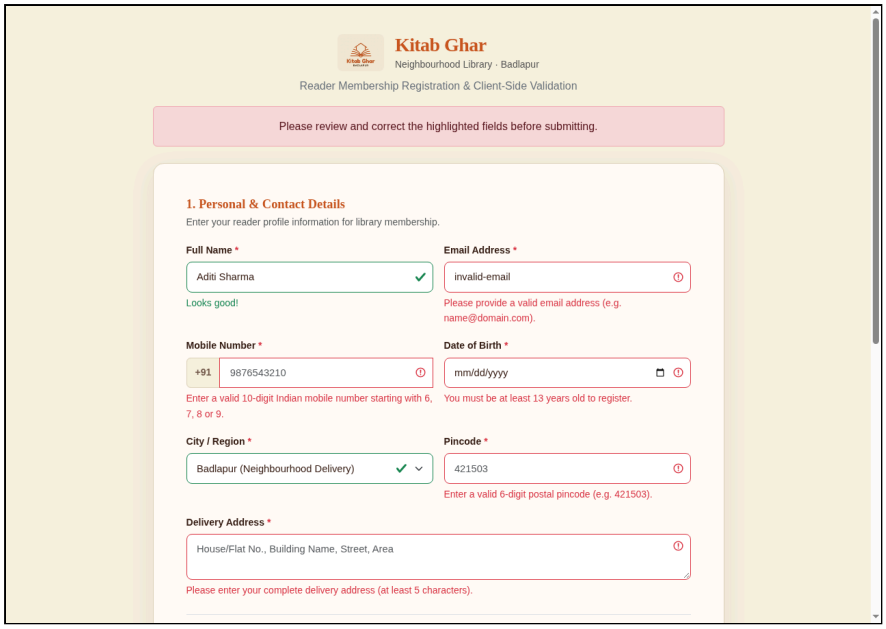


Figure 1.3: Email Format Validation Failure

This screenshot demonstrates client-side email format validation in action. Entering the malformed string 'invalid-email' without an '@' symbol or top-level domain fails the browser's type='email' constraint, triggering the invalid-feedback prompt while valid preceding inputs receive green is-valid indicators. This demonstrates the native HTML5 email validation combined with Bootstrap visual feedback, ensuring data quality before any network request is made.

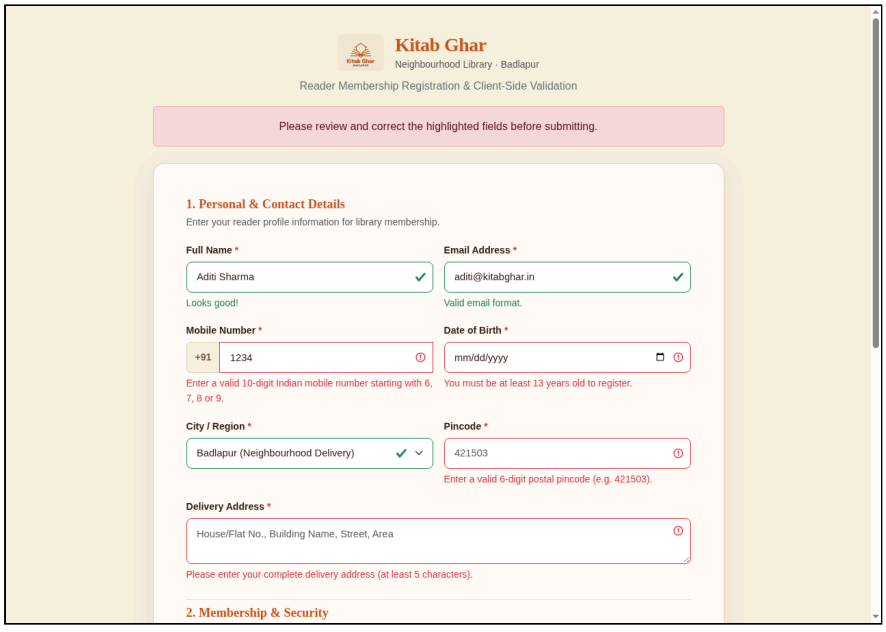


Figure 1.4: Mobile Number Pattern Validation Failure

This screenshot confirms the regular expression pattern validation for Indian contact numbers. The input '1234' violates the required 10-digit pattern [6-9][0-9]{9}, causing the input-group wrapper to turn red and display explicit instructions to enter a valid 10-digit Indian phone number starting with 6, 7, 8, or 9. The pattern attribute on the input element enforces this regex constraint natively in the browser, while JavaScript provides the custom feedback message for a user-friendly experience.

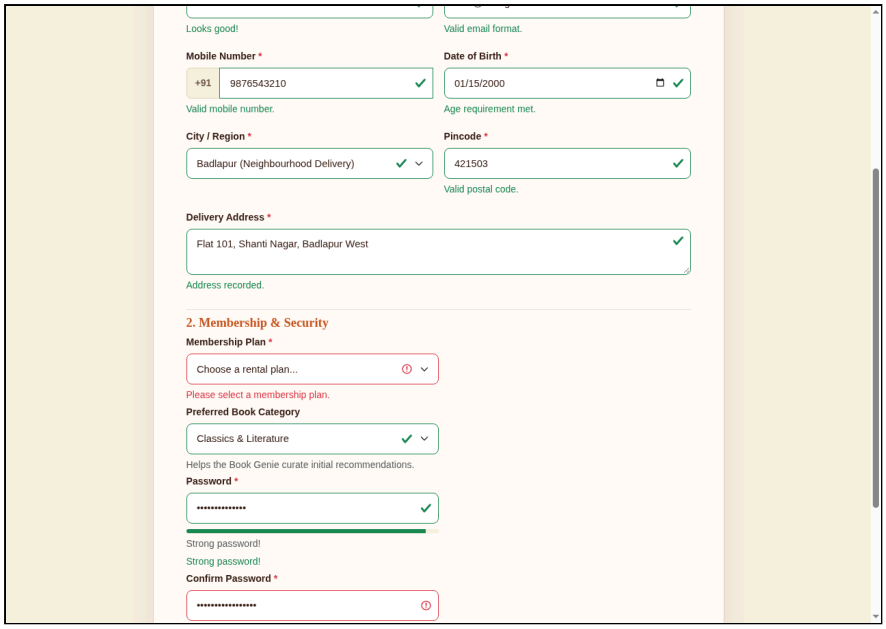


Figure 1.5: Password Strength Meter & Confirmation Mismatch Validation

This screenshot shows password validation and the custom mismatch rule. The dynamic progress bar updates in real time to reflect password strength based on character count, uppercase, lowercase, digit, and special character criteria. The confirmPassword field uses setCustomValidity('Passwords do not match') from the HTML5 Constraint Validation API to visibly reject mismatched credentials, demonstrating programmatic custom validation beyond standard HTML5 attributes.

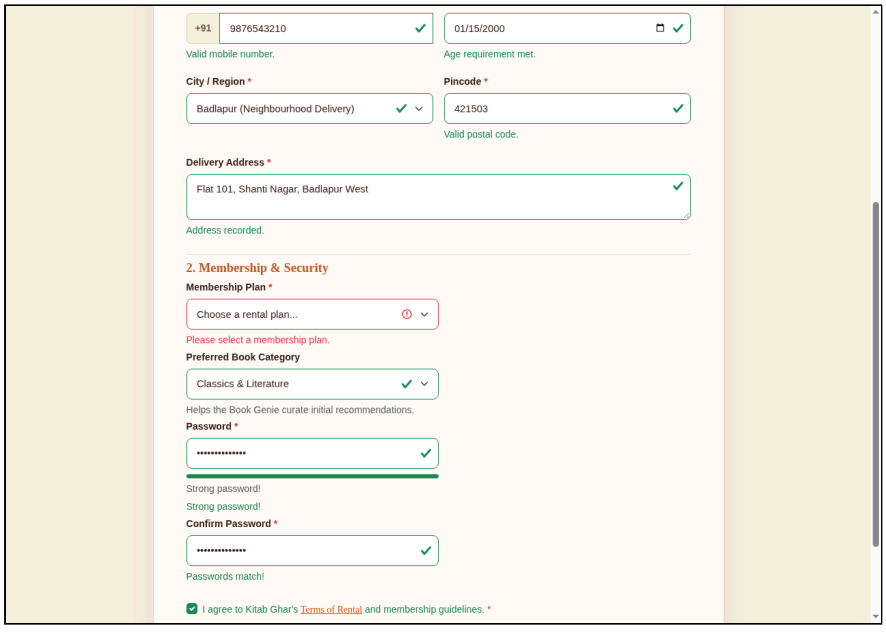


Figure 1.6: All Fields Validated Successfully

This screenshot proves successful client-side validation across all form controls. With valid reader credentials, Badlapur address, pincode (421503), date of birth, matching passwords, and accepted terms, every control displays the green is-valid border and a top-level alert-success banner confirms readiness. All Bootstrap validation states transitioned from is-invalid to is-valid, demonstrating the complete validation lifecycle from error detection through correction to successful submission.

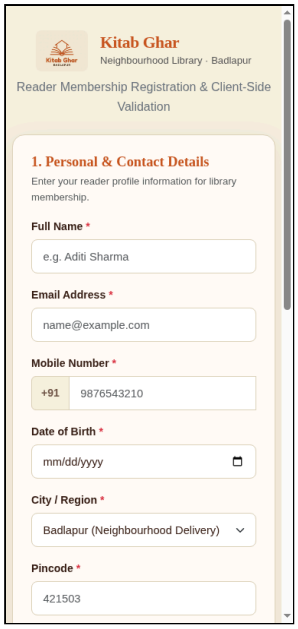


Figure 1.7: Responsive Mobile Layout (375px Viewport)

This screenshot shows the Kitab Ghar registration interface rendered on a mobile viewport (375px width). The Bootstrap 12-column grid automatically collapses the dual-column layout into a single, stacked column, ensuring touch targets and labels remain clear without horizontal scrolling. This confirms the responsive breakpoint behaviour of the col-md-6 grid classes, which stack below 768px viewport width, satisfying the mobile-first design requirement.

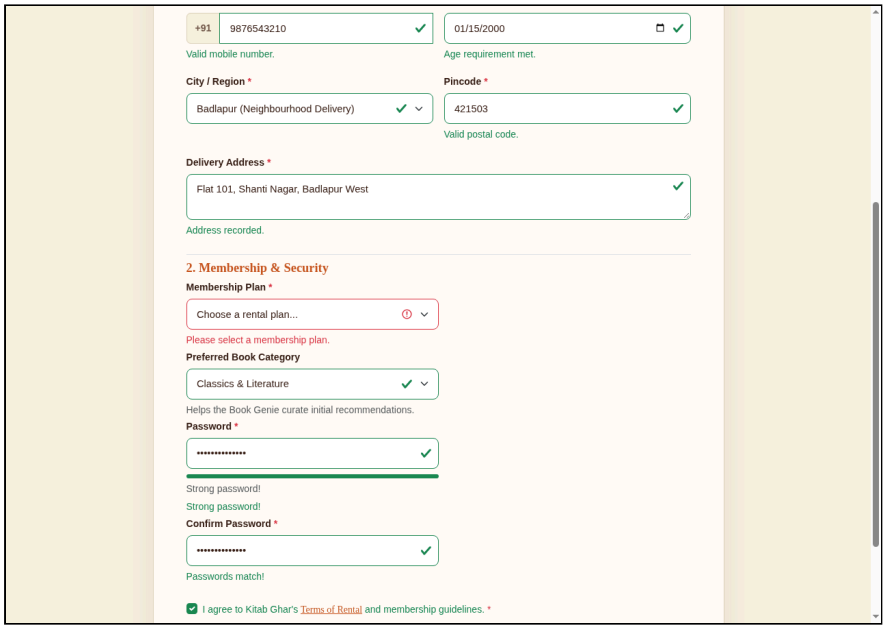


Figure 1.8: Bootstrap Registration Confirmation Modal

This screenshot shows the Bootstrap modal window rendered upon successful form submission. The JavaScript submit handler prevents standard page navigation via preventDefault(), compiles the validated reader data into a summary card, and triggers the modal overlay using Bootstrap's Modal API to simulate a completed membership registration. The modal displays the reader's name, email, selected membership type, and confirmation of successful validation.

Result:

A responsive registration form for Kitab Ghar was successfully designed using Bootstrap 5 and validated with JavaScript. All form controls adapt seamlessly across desktop and mobile screen resolutions. The Constraint Validation API, regular expressions, password complexity calculation, and custom mismatch logic reliably block invalid submissions and provide immediate visual guidance.

The validation sequence showed that invalid inputs were stopped before submission, while a valid reader record produced the expected confirmation state. The mobile check also confirmed that the form remained usable without horizontal overflow.

Conclusion:

The practical successfully demonstrated client-side form validation using Bootstrap 5 and modern JavaScript. The integration of HTML5 semantic constraints with custom JavaScript business logic guarantees data integrity before server submission.

The use of Bootstrap's responsive grid system ensured consistent usability across desktop and mobile devices, satisfying real-world web design requirements.

This experiment establishes a reusable registration workflow in which HTML5 constraints, JavaScript business rules, and a responsive Bootstrap layout work together before data is submitted.